AI Video Editing Platform: Functional & Technical Specification

Version 0.1 (draft). Stack: React, Node.js/Express, MongoDB, FFmpeg, BullMQ/Redis, AWS S3, Gemini/OpenAI API.

1. Purpose and scope

This document is the master blueprint for an AI-powered video editor where users describe edits in natural language and the system renders them. It defines three contracts:

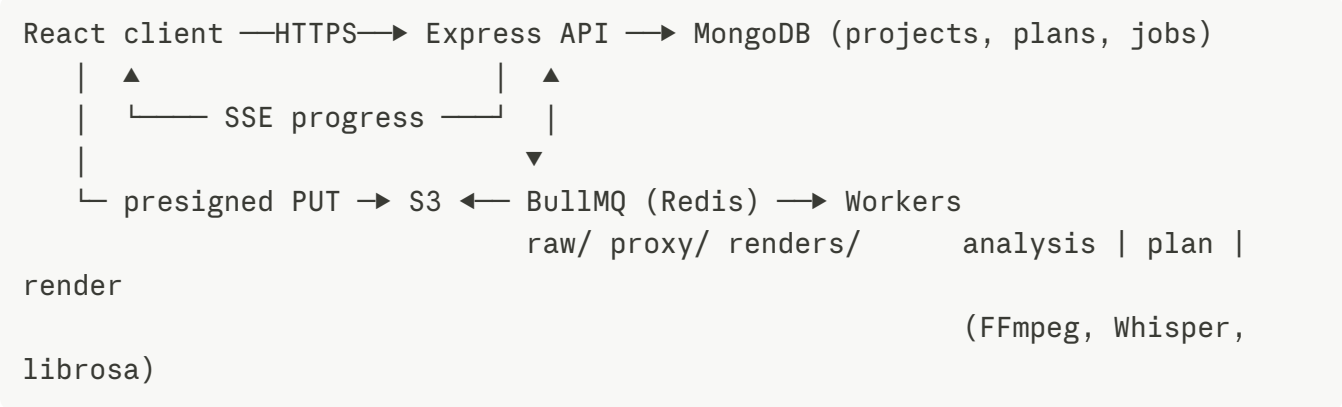
1. How a prompt becomes a validated JSON **Edit Plan** (Section 3).
2. How that plan moves through S3, Node.js worker queues and FFmpeg to a finished export (Section 4).
3. How the React client holds timeline, playback and canvas overlay state (Section 5).

Design principles

- The LLM outputs data, never commands. Only deterministic code builds FFmpeg arguments.
- Every edit is a versioned, reversible plan. The source file is never modified.
- Preview fast (480p proxy), render slow (final export), and use one shared compiler for both.
- Everything the client displays must be derivable from the Edit Plan.

In scope (MVP): upload, analysis, prompt to plan, proxy preview, final export, iterative chat edits. **Out of scope:** real-time collaboration, mobile editing, custom model training.

2. System architecture



Component	Responsibility
React client	Prompt box, timeline, preview player, overlay canvas, export dialog
Express API	Auth, project CRUD, presigned URLs, plan endpoints, job status (SSE)
Analysis worker	Transcription, silence/scene/beat detection, proxy generation
Planner service	Builds LLM context, calls LLM, validates and normalizes the plan
Render worker	Compiles plan to FFmpeg, runs it, uploads output, reports progress
MongoDB	Source of truth for plans, assets, jobs
S3	Raw uploads, proxies, subtitle files, music library, final renders
Redis	BullMQ queues, job progress pub/sub

3. AI-to-JSON command mapping

3.1 Flow

```
prompt + analysis → context builder → LLM (structured output)
   → Zod validation → normalizer (clamp, snap, resolve refs)
   → save EditPlan vN → return to client / enqueue render
```

The LLM never sees raw video. It receives: the user prompt, the current plan (if any), a compact analysis summary (transcript segments with word times, silence ranges, scene cuts, beat grid, duration, resolution, fps), and the allowed template and preset catalogs.

3.2 Edit Plan envelope

```
{
  "schemaVersion": "1.0",
  "projectId": "prj_8f2a",
  "baseAssetId": "ast_01",
  "version": 4,
  "parentVersion": 3,
  "global": { "aspect": "16:9", "fps": 30, "resolution": "1080p" },
  "layers": [ /* ordered array of layer objects, see 3.3 */ ],
  "meta": { "prompt": "make it cinematic", "model": "gemini", "createdAt":
"2026-11-04T10:00:00Z" }
}
```

Every layer shares this base shape:

```
{
  "id": "lyr_a1",
  "layer": "intro",
  "enabled": true,
  "timebase": "output",
  "range": { "start": 0, "end": 5 },
  "params": { }
}
```

Timebase rule. `source` means times refer to the original video (cuts, subtitles, text, transitions between source segments). `output` means times refer to the final timeline (intro, outro, music). The compiler owns conversion (Section 5.3 has the formula).

3.3 Layer types and parameters

layer	timebase	params (all clamped)
`cut`	source	`reason`: silence / filler / user / lowlight; `padding`: 0 to 0.3s
`speed`	source	`factor`: 0.25 to 4; `keepPitch`: bool
`color`	global	`preset`: cinematic / warm / cool / bw / vivid / faded; `brightness` -0.3 to 0.3; `contrast` 0.7 to 1.5; `saturation` 0 to 2; `lut`: catalog id; `vignette` 0 to 1; `grain` 0 to 0.5

layer	timebase	params (all clamped)
`subtitle`	source	`style`: classic / word_pop / karaoke / minimal; `font`: catalog; `size`: 24 to 96; `position`: bottom / center / top; `primaryColor`, `highlightColor`: hex; `maxWordsPerLine`: 2 to 8
`text`	source or output	`content` (max 120 chars); `animation`: fade / slide_up / typewriter / scale; `position`: {x,y} 0 to 1; `font`, `size`, `color`
`transition`	source	`type`: fade / dissolve / wipeleft / slideup / zoom; `duration`: 0.2 to 1.5; `at`: source time
`intro` / `outro`	output	`template`: catalog id; `duration`: 2 to 10; `text`, `subtext`; `theme`: hex palette
`overlay`	either	`asset`: catalog id (lower-third, logo, particle, lottie); `position`, `scale`, `opacity`
`music`	output	`trackId` or `mood`; `volume` 0 to 1; `duckUnderSpeech`: bool; `fadeIn`, `fadeOut`: seconds; `beatSync`: bool
`audio`	global	`normalizeLufs`: -23 to -14; `denoise`: bool

3.4 Intent mapping examples

User phrase	Layer(s) produced	Payload (abridged)
"Add an intro"	`intro`	`{template:"cinematic", duration:5}` (defaults applied if unspecified)
"Make it cinematic"	`color`	`{preset:"cinematic", contrast:1.15, saturation:0.9, lut:"teal_orange", vignette:0.3}`
"Remove the silent parts"	`cut` per range	one layer per silence longer than 0.6s from analysis
"Add animated subtitles"	`subtitle`	`{style:"word_pop", position:"bottom"}`
"Smooth transitions"	`transition`	`{type:"dissolve", duration:0.5}` at each scene cut

User phrase	Layer(s) produced	Payload (abridged)
"Use upbeat music"	`music`	`{mood:"upbeat", duckUnderSpeech:true, beatSync:true}`
"Make subtitles bigger"	patch	`replace /layers/lyr_s1/params/size`
"Faster pacing"	`cut` + `speed`	tighter silence threshold, `speed.factor` 1.1

3.5 Worked example

Prompt: *"Make this video cinematic, improve the colors, add animated subtitles, use smooth transitions, and create an engaging intro."*

```
{
  "layers": [
    { "id": "lyr_c1", "layer": "color", "enabled": true, "timebase":
"source",
      "params": { "preset": "cinematic", "contrast": 1.15, "saturation": 0.9,
                  "lut": "teal_orange", "vignette": 0.3 } },
    { "id": "lyr_s1", "layer": "subtitle", "enabled": true, "timebase":
"source",
      "params": { "style": "word_pop", "font": "Montserrat", "size": 56,
                  "position": "bottom", "highlightColor": "#FFD84D" } },
    { "id": "lyr_t1", "layer": "transition", "enabled": true, "timebase":
"source",
      "range": { "start": 12.4, "end": 12.9 },
      "params": { "type": "dissolve", "duration": 0.5, "at": 12.4 } },
    { "id": "lyr_i1", "layer": "intro", "enabled": true, "timebase":
"output",
      "range": { "start": 0, "end": 5 },
      "params": { "template": "cinematic", "duration": 5, "text": "My Trip to
Goa" } }
  ]
}
```

Vague phrases ("engaging") resolve to catalog defaults. Missing text for an intro is derived from the transcript or project title, and the assumption is recorded in `meta.assumptions` so the UI can surface it.

3.6 Patch operations (follow-up prompts)

Follow-up prompts return RFC 6902-style patches against the current plan, not a full regenerated plan. This preserves everything the user did not mention.

```
{ "patch": [
  { "op": "replace", "path": "/layers/lyr_s1/params/size", "value": 72 },
  { "op": "add", "path": "/layers/-", "value": { "id": "lyr_m1", "layer":
"music", "params": { "mood": "calm_cinematic" } } },
  { "op": "remove", "path": "/layers/lyr_t1" }
] }
```

The server applies the patch, re-validates the whole plan, and saves a new version with `parentVersion` set.

3.7 Validation and normalization

1. **Schema:** Zod discriminated union on `layer`. Unknown fields are stripped, unknown layer types rejected.
2. **Clamping:** numeric params forced into the ranges in 3.3.
3. **Reference resolution:** template, font, LUT, track IDs must exist in the catalog, otherwise fall back to the nearest default.
4. **Time sanity:** ranges inside asset duration, no overlapping cuts (merged), transitions not on cut boundaries shorter than their duration.
5. **Budget:** max 200 layers, max 120 characters per text, intro/outro under 10s.
6. **On failure:** one automatic repair retry sending the validation errors back to the LLM, then return a friendly error.

3.8 LLM contract

- Use structured output / function calling with the JSON schema, temperature 0.2.
- System prompt states: output only schema-valid JSON, never invent asset IDs, prefer catalog defaults, never remove more than 40 percent of footage without explicit user wording.
- User content is wrapped as data. Transcript text is untrusted and must not be treated as instructions.

4. FFmpeg processing pipeline

4.1 S3 layout

```
s3://bucket/
  users/{uid}/projects/{pid}/
    raw/{assetId}.mp4            original upload (immutable)
    proxy/{assetId}_480p.mp4     analysis and preview source
    audio/{assetId}.wav          16 kHz mono for Whisper
    subs/{planVersion}.ass       generated subtitle file
    previews/{planVersion}.mp4   480p plan preview
    renders/{jobId}.mp4          final export
  library/music/  library/luts/  library/templates/  library/fonts/
```

4.2 End-to-end lifecycle

Step	Actor	Action	State
1	Client	`POST /uploads` returns presigned multipart URLs, client uploads directly to S3	`uploading`
2	API	On completion webhook, create Asset, enqueue `analysis` job	`queued`
3	Analysis worker	Download raw, run ffprobe, create proxy and WAV, transcribe, silencedetect, scene detect, beat detect. Save to MongoDB.	`analyzing`
4	Client	`POST /projects/:id/prompt`	`planning`
5	Planner	LLM call, validate, save EditPlan vN	`planned`
6	Render worker	Compile plan, render 480p preview, upload	`previewing`
7	Client	Plays preview, user approves or sends another prompt (back to 4)	`ready`
8	Client	`POST /plans/:id/export`	`rendering`
9	Render worker	Full-res render from raw, upload to `renders/`	`done`
10	API	Returns time-limited presigned GET URL	`done`

4.3 Queues and workers

Queue	Concurrency	Timeout	Retries
`analysis`	2 per node	20 min	2, exponential backoff
`preview`	3 per node	5 min	1
`render`	1 per CPU group (GPU optional)	60 min	1

Job data holds only IDs (`projectId` , `planVersion` , `jobId`), never plan bodies, so the worker always reads the latest validated plan from MongoDB. Workers are stateless, use a per-job temp directory (`/tmp/job-{id}`), and delete it in a `finally` block. Rendering workers should run in containers with CPU and memory limits and no network access except S3.

4.4 Plan compiler

The compiler (`compiler.js`) is a pure function: `compile(plan, analysis, profile) -> { args, assFiles, inputs }` . It runs in five passes:

1. **Segment build:** apply `cut` layers to produce keep-segments in source time.
2. **Per-segment video chain:** `trim` , `setpts` , `speed` (`setpts=PTS/f`), then color filters.
3. **Join:** `xfade` between segments where `transition` layers apply, otherwise `concat` .
4. **Overlays:** subtitles (ASS), text, overlays, intro/outro via `overlay` or `concat` .
5. **Audio:** matching `atrim` / `atempo` / `acrossfade` , then music mix, ducking, loudness.

Layer	FFmpeg approach
`cut`	`trim` + `setpts=PTS-STARTPTS` , `atrim` + `asetpts` , joined by `concat`
`speed`	`setpts=PTS/f` , `atempo=f` (chain for factors above 2 or below 0.5)
`color`	`eq=contrast:brightness:saturation` , `colorbalance` , `curves` , `lut3d=file.cube` , `vignette` , `noise=alls=12:allf=t`
`subtitle`	Generate `.ass` with per-word `\k` karaoke tags and `\fad` , burn with `subtitles=file.ass:fontsdir=...`
`text`	ASS event (preferred) or `drawtext` with `enable='between(t,a,b)'`

Layer	FFmpeg approach
`transition`	`xfade=transition=fade:duration=0.5:offset=X`, paired `acrossfade`
`intro` / `outro`	Template pre-rendered clip (Remotion or Lottie) with text injected, normalized to project fps/size, then `concat`
`overlay`	`overlay=x:y:enable='between(t,a,b)'` on a looped/alpha input
`music`	`afade`, `volume`, `sidechaincompress=threshold=0.05:ratio=8:attack=20:release=400` keyed from dialogue, `amix`
`audio`	`loudnorm=I=-16:TP=-1.5:LRA=11`, optional `afftdn`

4.5 Example invocation

Always `child_process.spawn('ffmpeg', argsArray)`, never a shell string. Simplified result for one cut, color and subtitles:

```
ffmpeg -y -i raw.mp4 -i music.mp3 \
 -filter_complex "\
 [0:v]trim=0:4.2,setpts=PTS-STARTPTS[v0]; \
 [0:v]trim=5.9:60,setpts=PTS-STARTPTS[v1]; \
 [0:a]atrim=0:4.2,asetpts=PTS-STARTPTS[a0]; \
 [0:a]atrim=5.9:60,asetpts=PTS-STARTPTS[a1]; \
 [v0][v1]concat=n=2:v=1:a=0[vc]; [a0][a1]concat=n=2:v=0:a=1[ac]; \
 [vc]eq=contrast=1.15:saturation=0.9,lut3d=teal_orange.cube,vignette[vg]; \
 [vg]subtitles=subs.ass[vout]; \
 [1:a]volume=0.35[m]; [m][ac]sidechaincompress[md]; [md]
[ac]amix=inputs=2:normalize=0[aout]" \
 -map "[vout]" -map "[aout]" -c:v libx264 -crf 20 -preset medium -c:a aac -
b:a 192k \
 -movflags +faststart -progress pipe:1 out.mp4
```

4.6 Output profiles

Profile	Resolution	Encode	Use
`preview`	854x480	libx264 `-preset veryfast -crf 28`, 24 to 30 fps	Plan review, generated per plan version

Profile	Resolution	Encode	Use
`final_1080p`	1920x1080	libx264 `-preset medium -crf 18`, AAC 192k	Default export
`final_4k`	3840x2160	libx264 or hevc, `-crf 18`, requires larger worker class	Paid tier

Previews render from the proxy, finals from the raw file. Because both use the same compiler, any difference should be resolution only.

4.7 Progress, errors and retries

- Parse `-progress pipe:1` (`out_time_ms`) divided by expected output duration to get percent. Publish to Redis channel `job:{id}`, and the API relays it over SSE.
- Classify failures: `INPUT_CORRUPT` (no retry), `FILTER_ERROR` (no retry, log plan and args), `OOM_OR_TIMEOUT` (retry once on a larger worker), `S3_TRANSIENT` (retry with backoff).
- A failed render keeps the plan intact so the user can fix the prompt, never leaving the project in a half-state.
- Delete previews older than 7 days and abandoned uploads after 24 hours with S3 lifecycle rules.

4.8 Security

- No LLM output or user text is interpolated into shell strings. Text is written to ASS files with escaping of `{`, `}`, and `\`.
- File paths and catalog assets come only from server-side lookups by ID.
- Presigned URLs expire in 15 minutes (upload) and 1 hour (download). Enforce max file size and MIME sniffing with ffprobe before processing.
- Rate limit prompts per user and cap LLM tokens per project.
- Music and templates come only from the licensed library.

5. Frontend state management

5.1 Approach

Concern	Tool
Editor state (timeline, playback, selection)	Zustand with Immer, sliced stores

Concern	Tool
Server data (project, plans, assets)	TanStack Query
Job progress	`EventSource` (SSE) feeding the store
Overlay drawing	Canvas 2D (or OffscreenCanvas in a worker)
Undo/redo	Plan version stack, not local patches

Principle: **the Edit Plan is the single source of truth.** The timeline, overlays and inspector are all derived selectors over `plan.layers`.

5.2 Store shape

```
interface EditorState {
  project: { id: string; status: ProjectStatus; assetId: string; duration:
number };
  plan: {
    current: EditPlan;          // validated plan from server
    draft: EditPlan | null;     // optimistic local tweaks before server
confirm
    version: number;
    history: number[];          // version ids for undo/redo
    cursor: number;
  };
  timeline: {
    zoom: number;               // pixels per second
    scrollX: number;
    selectedLayerIds: string[];
    snap: boolean;
    tracks: TrackVM[];          // derived: video, subtitle, text, fx, music
  };
  playback: {
    status: 'idle' | 'loading' | 'playing' | 'paused' | 'seeking' | 'ended';
    currentTime: number;        // OUTPUT seconds
    duration: number;           // output duration after cuts/intro
    rate: number;
    volume: number;
    muted: boolean;
  };
  preview: {
    mode: 'proxy' | 'live';     // rendered preview vs live overlay
approximation
    url: string | null;
    planVersion: number | null; // which version the url reflects
```

```
    stale: boolean;             // true if plan changed after preview
rendered
  };
  jobs: Record<string, { type: 'preview'|'render'|'analysis'; percent:
number; state: string; error?: string }>;
  chat: { messages: ChatMessage[]; pending: boolean };
}
```

5.3 Timeline model and time mapping

The user scrubs in **output time**, but layers are stored partly in **source time**. A pure function pair keeps them consistent and is unit tested:

```
// keepSegments: [{srcStart, srcEnd, speed}] from cut/speed layers, in order
function srcToOut(t: number, segs: Seg[], introDur: number, overlaps:
number[]): number | null {
  let out = introDur;
  for (let i = 0; i < segs.length; i++) {
    const s = segs[i];
    if (t >= s.srcStart && t < s.srcEnd) return out + (t - s.srcStart) /
s.speed;
    out += (s.srcEnd - s.srcStart) / s.speed - (overlaps[i] ?? 0);
  }
  return null; // inside a removed cut
}
function outToSrc(t: number, segs: Seg[], introDur: number, overlaps:
number[]): number | null { /* inverse */ }
```

Tracks derived for rendering: **Video** (keep-segments as clips, cuts shown as ripples), **Subtitles** (one block per caption line), **Text/FX** (text, overlay, intro, outro), **Music** (waveform plus duck envelope), **Markers** (scene cuts, beats, silences from analysis, toggleable).

5.4 Playback engine

- A single `<video>` element is the master clock. The store never runs its own timer for time; it samples `video.currentTime` in a `requestAnimationFrame` loop while playing and writes `playback.currentTime` (throttled to about 30 Hz).
- Seek: set `status='seeking'`, assign `video.currentTime`, resume on the `seeked` event.
- When the preview is **stale** (plan changed, render not finished), the UI shows a banner and either plays the old preview or falls back to live mode (5.5).

- Audio from the music layer is mixed only in rendered previews. In live mode it is a separate `<audio>` element kept within 80 ms of the video clock, corrected by nudging `playbackRate`.

5.5 Real-time canvas overlay preview

To give instant feedback before a render finishes, a `<canvas>` sits absolutely positioned over the video (same CSS size, `pointer-events` toggled for editing).

```
rAF tick
  t = playback.currentTime (output)
  active = selectActiveLayers(plan, t)        // memoized by layer id and t
bucket
  ctx.clearRect(...)
  for layer of active:
    drawers[layer.layer](ctx, layer, localProgress, viewport)
```

Layer	Live approximation
`subtitle` / `text`	Draw with same font, size and animation easing as ASS output (shared animation constants package)
`color`	CSS `filter` on the video element (`contrast()` , `saturate()` , `brightness()`) plus vignette gradient on canvas
`overlay` / `intro`	Lottie via `lottie-web` canvas renderer or first-frame poster
`transition`	Not simulated live, shown as a marker; accurate only in rendered preview

Rules: coordinates are normalized 0 to 1 so overlays scale with the player; fonts load through `FontFace` before first draw; the canvas redraws only when `t` or the plan changes while paused. Live mode is explicitly labeled "approximate", and rendered preview is the source of truth.

5.6 Prompt, plan and server sync

```
user sends prompt
  chat.pending = true
  POST /prompt ⟶ plan v(N+1) returned
  plan.current = v(N+1); history push; preview.stale = true
  POST /preview (auto) ⟶ jobs[jobId] via SSE percent updates
  on done: preview.url = signed url; preview.planVersion = N+1; stale = false
```

Direct manipulation (drag a transition, resize a subtitle) updates `plan.draft` immediately for the live overlay, then debounces 600 ms and sends a patch. The server response replaces `draft`. On conflict (409), refetch and show a toast.

5.7 Undo and redo

Undo moves `cursor` back in `plan.history` and sets `plan.current` to that stored version, then marks the preview stale (or reuses a cached preview for that version if one exists). Redo moves forward. A new edit after undo truncates forward history locally while the server keeps all versions.

5.8 Performance targets

- Timeline virtualized: render only clips within viewport plus a 1-screen buffer.
- Selectors memoized per slice, with `zustand/shallow` equality.
- Canvas draw under 4 ms per frame at 1080p CSS size.
- Scrubbing latency under 100 ms using the 480p proxy.

6. Data models (MongoDB)

Collection	Key fields
`users`	`_id`, email, plan tier, quota
`projects`	`_id`, userId, title, status, baseAssetId, currentPlanVersion
`assets`	`_id`, projectId, type, s3Key, proxyKey, duration, width, height, fps, codec, `analysis` (embedded)
`analysis` (embedded)	`transcript[{start,end,text,words[]}]`, `silences[]`, `sceneCuts[]`, `beats[]`, `loudness`
`editPlans`	`_id`, projectId, version, parentVersion, plan (JSON), prompt, createdBy

Collection	Key fields
`renderJobs`	`_id`, projectId, planVersion, profile, type, state, percent, outputKey, error, timings
`catalog`	`kind` (template/lut/font/music), id, name, s3Key, metadata

Indexes: `editPlans(projectId, version)` unique; `renderJobs(projectId, state)`; `assets(projectId)`.

7. API surface

Method and path	Purpose
`POST /api/uploads`	Start multipart upload, returns presigned URLs
`POST /api/uploads/:id/complete`	Finalize, create asset, enqueue analysis
`GET /api/projects/:id`	Project, asset metadata, analysis summary
`POST /api/projects/:id/prompt`	Body `{prompt}`. Returns new plan version (or patch result)
`GET /api/projects/:id/plans/:v`	Fetch a plan version
`PATCH /api/projects/:id/plans/:v`	Apply manual patch, returns new version
`POST /api/projects/:id/preview`	Enqueue preview render for a plan version
`POST /api/projects/:id/export`	Enqueue final render with profile
`GET /api/jobs/:id/events`	SSE stream: `progress`, `state`, `error`, `done`
`GET /api/jobs/:id/download`	Presigned download URL

8. Non-functional requirements

Area	Target
Analysis time	Under 0.5x realtime for a 10-minute video
Plan generation	Under 15 s p95
Preview render	Under 0.5x realtime at 480p

Area	Target
Availability	99.5 percent API, queues drain after outage
Upload size	Up to 5 GB, 60 minutes (MVP)
Observability	Structured logs with `projectId` / `jobId`, queue depth and failure-rate alerts
Privacy	Encrypted S3 buckets, per-user key prefixes, delete-on-request purge
Cost control	Cache analysis, cap LLM tokens, skip re-render when plan hash is unchanged

9. Milestones and acceptance criteria

Phase	Deliverable	Acceptance test
1	Upload, analysis, auto-captions burned in	5-minute video yields captioned preview with word timing within 150 ms
2	Silence cuts and color presets	Cut result has no gaps over 0.6 s; preset visibly changes frames
3	LLM planner with schema and validation	95 percent of a 50-prompt test set yields valid plans on first try
4	Transitions and intro/outro templates	`xfade` and intro render without A/V desync (under 40 ms drift)
5	Music selection, beat sync, ducking	Cuts snap to beats within 60 ms; speech stays intelligible (ducking at least 8 dB)
6	Chat patch edits, undo/redo, live overlay	"Make subtitles bigger" changes only that param; undo restores prior version

10. Open questions

- Intro and outro templates: Remotion server rendering or pre-built Lottie/After Effects clips?
- Whisper API vs self-hosted `faster-whisper` once volume grows?
- GPU encoding (NVENC) for the final render tier?
- Multi-clip projects: extend Edit Plan with a `clips[]` array in v1.1?